

LAPORAN PRAKTIKUM

STRUKTUR DATA

“ALGORITMA SORTING”



disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillah rabbil ‘alamin, segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga saya dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “*ALGORITMA SORTING*”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam wawasan saya mengenai penerapan konsep struktur data, khususnya *Algoritma Sorting* dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan data yang bersifat dinamis.

Saya menyampaikan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Algoritma dan Pemrograman, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga saya sampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan dan keterbatasan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum di masa mendatang. Meskipun demikian, saya berharap laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	2
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	4
2.1 Program <i>ShellSort</i>	4
2.2 Program <i>QuickSort</i>	10
2.3 Program <i>MergeSort</i>	16
2.4 Program <i>BubbleSortGUI</i>	23
2.5 Program <i>MergeSortGUI</i>	39
BAB III KESIMPULAN.....	56
3.1 Kesimpulan	56
DAFTAR PUSTAKA	58

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran Struktur Data, mahasiswa tidak hanya mempelajari konsep dasar pemrograman, tetapi juga mempelajari berbagai algoritma yang digunakan untuk mengelola dan mengolah data secara efisien. Salah satu materi penting dalam struktur data adalah algoritma sorting atau pengurutan data.

Sorting merupakan proses mengurutkan data berdasarkan urutan tertentu, seperti dari nilai terkecil ke terbesar atau sebaliknya. Proses pengurutan data sangat penting karena dapat mempermudah pencarian, pengelompokan, dan pengolahan data dalam berbagai aplikasi komputer. Oleh karena itu, pemahaman mengenai algoritma *sorting* menjadi salah satu kompetensi yang harus dimiliki oleh mahasiswa dalam mempelajari struktur data.

Pada praktikum ini digunakan beberapa algoritma sorting, yaitu *Insertion Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort*. *Insertion Sort* bekerja dengan cara menyisipkan elemen pada posisi yang tepat dalam bagian array yang telah terurut. *Shell Sort* merupakan pengembangan dari *Insertion Sort* yang menggunakan jarak tertentu (*gap*) untuk mempercepat proses pengurutan. *Quick Sort* bekerja dengan memilih sebuah *pivot* kemudian membagi data menjadi beberapa bagian yang lebih kecil untuk diurutkan secara rekursif. Sedangkan *Merge Sort* menggunakan metode *divide and conquer* dengan membagi *array* menjadi beberapa bagian, kemudian menggabungkannya kembali dalam keadaan terurut.

Selain program berbasis console, praktikum ini juga mengimplementasikan *Java GUI (Swing)* melalui program *Bubble Sort GUI* dan *Merge Sort GUI* untuk menampilkan proses pengurutan data secara visual dan interaktif. Program *GUI* memungkinkan pengguna

memasukkan data *array* secara manual serta mengamati setiap langkah pengurutan secara bertahap sehingga lebih mudah memahami cara kerja algoritma *sorting*.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar algoritma *sorting*, mengetahui karakteristik masing-masing algoritma, serta mampu mengimplementasikan dan memvisualisasikan proses pengurutan data menggunakan bahasa pemrograman Java.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma *sorting* dalam struktur data.
2. Mengetahui cara kerja algoritma *Insertion Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort*.
3. Mempelajari implementasi algoritma *sorting* menggunakan bahasa Java.
4. Mengimplementasikan proses pengurutan data menggunakan *array*.
5. Membandingkan beberapa metode pengurutan data berdasarkan cara kerjanya.
6. Membuat program *GUI* menggunakan *Java Swing* untuk visualisasi proses *sorting*.
7. Menampilkan langkah-langkah proses *sorting* secara bertahap pada program *GUI*.
8. Melatih kemampuan dalam membuat program struktur data dan algoritma menggunakan Java.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman tentang algoritma *sorting* dalam struktur data.
2. Membantu memahami proses pengurutan data menggunakan *Insertion Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort*.

3. Melatih kemampuan membuat program Java berbasis *console* dan *GUI (Swing)*.
4. Meningkatkan kemampuan logika dan analisis dalam proses pengurutan data.
5. Membantu memahami implementasi algoritma *divide and conquer* pada *Quick Sort* dan *Merge Sort*.
6. Membantu memahami visualisasi langkah-langkah proses sorting secara interaktif.
7. Membantu memahami penggunaan *array* dalam proses pengolahan data.
8. Melatih kemampuan membuat program interaktif menggunakan *Java GUI*.

BAB II

PEMBAHASAN

2.1 Program *ShellSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan8_2511532025;

public class ShellSort_2511532025 {
    public static void ShellSort_2025(int[] A_2025) {
        int n_2025 = A_2025.length;
        int gap_2025 = n_2025/2;
        while (gap_2025 > 0) {
            for (int i_2025 = gap_2025; i_2025 < n_2025;
i_2025++) {
                int temp_2025 = A_2025[i_2025];
                int j_2025 = i_2025;
                while (j_2025 >= gap_2025 &&
A_2025[j_2025 - gap_2025] > temp_2025) {
                    A_2025[j_2025] = A_2025[j_2025 -
gap_2025];
                    j_2025 = j_2025 - gap_2025;
                }
                A_2025[j_2025] = temp_2025;
            }
            gap_2025 = gap_2025/2;
        }
    }

    public static void main(String[] args) {
        int[] data_2025 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_2025(data_2025);

        ShellSort_2025(data_2025);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_2025(data_2025);
    }

    public static void printArray_2025(int[] arr_2025) {
        for (int i_2025 : arr_2025) System.out.print(i_2025
+ " ");
        System.out.println();
    }
}
```

Kode Program 2.1 Program *ShellSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Shell Sort*. Algoritma ini merupakan pengembangan dari

Insertion Sort yang bekerja dengan membandingkan elemen-elemen yang berjarak tertentu (*gap*). Nilai *gap* akan terus diperkecil hingga bernilai 1. Dengan cara ini, data dapat diurutkan lebih cepat dibandingkan *Insertion Sort* pada data yang berukuran besar.

Method ShellSort_2025() digunakan untuk melakukan proses pengurutan data. Program terlebih dahulu menentukan nilai *gap* sebesar setengah dari panjang *array*. Selanjutnya dilakukan perulangan untuk membandingkan elemen-elemen yang dipisahkan oleh jarak *gap* tersebut. Jika ditemukan elemen yang lebih besar dari elemen yang sedang diperiksa, maka elemen akan digeser hingga ditemukan posisi yang sesuai.

Pada *method main()*, program membuat *array* berisi data 3, 10, 4, 6, 8, 9, 7, 2, 1, 5. Selanjutnya program menampilkan isi *array* sebelum diurutkan, memanggil *method ShellSort_2025()* untuk melakukan pengurutan data, dan menampilkan hasil *array* setelah terurut.

2.1.1 Langkah Kerja Program *ShellSort*

1. Membuat *class ShellSort_2511532025*

```
public class ShellSort_2511532025 {
```

Kode Program 2.2 *Class IntertionSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Shell Sort*.

2. Membuat *method ShellSort_2511532025*

```
public static void ShellSort_2025(int[] A_2025) {
```

Kode Program 2.3 *Method ShellSort*

Method ini digunakan untuk melakukan proses pengurutan *array* menggunakan algoritma *Shell Sort*.

3. Mengambil panjang *array*

```
int n_2025 = A_2025.length;
```

Kode Program 2.4 Panjang *Array*

Syntax ini digunakan untuk mengambil jumlah elemen pada *array*.

4. Menentukan nilai *gap* awal

```
int gap_2025 = n_2025/2;
```

Kode Program 2.5 Menentukan *Gap* Awal

Syntax ini digunakan untuk menentukan jarak awal antar elemen yang akan dibandingkan.

5. Melakukan perulangan selama *gap* lebih besar dari 0

```
while (gap_2025 > 0) {
```

Kode Program 2.6 Perulangan *Gap*

Perulangan ini digunakan untuk menjalankan proses pengurutan selama nilai *gap* masih lebih besar dari 0.

6. Melakukan perulangan elemen *array*

```
for (int i_2025 = gap_2025; i_2025 < n_2025; i_2025++) {
```

Kode Program 2.7 Perulangan *Array*

Perulangan ini digunakan untuk memeriksa setiap elemen *array* berdasarkan nilai *gap* yang digunakan.

7. Menyimpan nilai sementara dan indeks

```
int temp_2025 = A_2025[i_2025];  
int j_2025 = i_2025;
```

Kode Program 2.8 Menyimpan Nilai Sementara

Syntax ini digunakan untuk menyimpan nilai yang akan dipindahkan dan menentukan posisi indeks saat ini.

8. Membandingkan dan menggeser elemen

```
while (j_2025 >= gap_2025 &&  
A_2025[j_2025 - gap_2025] > temp_2025) {  
    A_2025[j_2025] =  
A_2025[j_2025 - gap_2025];  
    j_2025 = j_2025 - gap_2025;  
}
```

Kode Program 2.9 Proses Perbandingan

Syntax ini digunakan untuk membandingkan elemen yang berjarak *gap*. Jika elemen sebelumnya lebih besar, maka elemen tersebut akan digeser ke kanan.

9. Menempatkan nilai pada posisi yang sesuai

```
A_2025[j_2025] = temp_2025;  
}
```

Kode Program 2.10 Menempatkan Nilai

Syntax ini digunakan untuk menempatkan nilai sementara pada posisi yang tepat setelah proses pergeseran selesai.

10. Mengurangi nilai *gap*

```
gap_2025 = gap_2025/2;
```

Kode Program 2.11 Mengurangi *Gap*

Syntax ini digunakan untuk memperkecil nilai *gap* hingga akhirnya bernilai 1 dan proses pengurutan selesai.

11. Membuat *method main*

```
public static void main(String[] args) {
```

Kode Program 2.12 *Method Main*

Method main() merupakan *method* utama yang digunakan untuk menjalankan program.

12. Membuat *array data*

```
int[] data_2025 = {3, 10, 4, 6, 8, 9, 7, 2,  
1, 5};
```

Kode Program 2.13 Membuat *Array Data*

Array ini digunakan sebagai data yang akan diurutkan menggunakan algoritma *Shell Sort*.

13. Menampilkan *array* sebelum diurutkan

```
System.out.print("Sebelum: ");  
printArray_2025(data_2025);
```

Kode Program 2.14 Menampilkan *Array Awal*

Syntax ini digunakan untuk menampilkan isi *array* sebelum dilakukan proses pengurutan.

14. Memanggil *method ShellSort_2025*

```
ShellSort_2025(data_2025);
```

Kode Program 2.15 Memanggil *Method Sorting*

Syntax ini digunakan untuk menjalankan proses pengurutan data.

15. Menampilkan *array* setelah diurutkan

```
System.out.print("Sesudah (Shell Sort): ");  
printArray_2025(data_2025);  
}
```

Kode Program 2.16 Menampilkan Hasil *Sorting*

Syntax ini digunakan untuk menampilkan hasil *array* setelah proses pengurutan selesai.

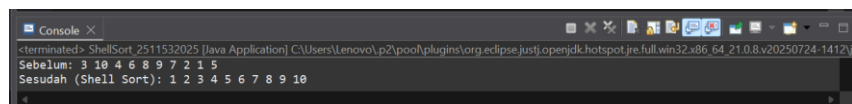
16. Membuat *method printArray_2025*

```
public static void printArray_2025(int[] arr_2025) {  
    for (int i_2025 : arr_2025)  
        System.out.print(i_2025 + " ");  
        System.out.println();  
    }  
}
```

Kode Program 2.17 *Method Print Array*

Method ini digunakan untuk menampilkan seluruh isi *array* ke layar.

2.1.2 Hasil Program *ShellSort*



Gambar 2.1 Hasil Program *ShellSort*

Berdasarkan hasil eksekusi program, algoritma *Shell Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi *array* adalah:

3 10 4 6 8 9 7 2 1 5

Setelah *method ShellSort_2025()* dijalankan, *array* berhasil diurutkan menjadi:

1 2 3 4 5 6 7 8 9 10

Hasil tersebut menunjukkan bahwa algoritma *Shell Sort* bekerja dengan cara membandingkan elemen-elemen yang dipisahkan oleh jarak tertentu (*gap*), kemudian menggeser elemen yang lebih besar

hingga ditemukan posisi yang sesuai. Nilai *gap* akan terus diperkecil sampai bernilai 1 sehingga seluruh elemen *array* tersusun secara terurut dari nilai terkecil ke terbesar.

2.2 Program *QuickSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan8_2511532025;

public class QuickSort_2511532025 {
    static void swap_2025(int[] arr_2025, int i_2025, int
j_2025) {
        int temp_2025 = arr_2025[i_2025];
        arr_2025[i_2025] = arr_2025[j_2025];
        arr_2025[j_2025] = temp_2025;
    }
    // Metode tambahan untuk mengatur pivot_2025 menggunakan
Median-of-Three
    static void medianOfThree_2025(int[] arr_2025, int
low_2025, int high_2025) {
        int mid_2025 = low_2025 + (high_2025 - low_2025)/2;

        // Urutkan elemen low, mid dan high
        if (arr_2025[low_2025] > arr_2025[mid_2025]) {
            swap_2025(arr_2025, low_2025, mid_2025);
        }
        if (arr_2025[low_2025] > arr_2025[high_2025]) {
            swap_2025(arr_2025, low_2025, high_2025);
        }
        if (arr_2025[mid_2025] > arr_2025[high_2025]) {
            swap_2025(arr_2025, mid_2025, high_2025);
        }
        swap_2025(arr_2025, mid_2025, high_2025);
    }
    static int partition_2025(int[] arr_2025, int low_2025, int
high_2025) {

        // Panggil fungsi medianOfThree sebelum menentukan
pivot_2025
        medianOfThree_2025(arr_2025, low_2025, high_2025);

        int pivot_2025 = arr_2025[high_2025]; // Sekarang
arr_2025[high] sudah berisi nilai median
        int i_2025 = (low_2025- 1);

        for (int j_2025 = low_2025; j_2025 <= high_2025 - 1;
j_2025++) {
            // Jika elemen saat ini lebih kecildari atau
sama dengan pivot_2025
            if (arr_2025[j_2025] < pivot_2025) {
                // Increment indeks elemen yang lebih
kecil
                i_2025++;
            }
        }
    }
}
```

```

        swap_2025(arr_2025, i_2025, j_2025);
    }
}
swap_2025(arr_2025, i_2025 + 1, high_2025);
return (i_2025 + 1);
}
static void quicksort_2025(int[] arr_2025, int low_2025,
int high_2025) {
    if (low_2025 < high_2025) {
        int pi_2025 = partition_2025(arr_2025,
low_2025, high_2025);
        quicksort_2025(arr_2025, low_2025, pi_2025 -
1);
        quicksort_2025(arr_2025, pi_2025 + 1,
high_2025);
    }
}
public static void printArr_2025(int[] arr_2025) {
    for(int i_2025 = 0; i_2025 < arr_2025.length;
i_2025++) {
        System.out.print(arr_2025[i_2025] + " ");
    }
    System.out.println();
}
public static void main(String[] args) {
    int[] arr_2025 = { 10, 7, 8, 9, 1, 5 };
    int N_2025 = arr_2025.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2025(arr_2025);

    quicksort_2025(arr_2025, 0, N_2025 - 1);

    System.out.print("Data terurut quicksort: ");
    printArr_2025(arr_2025);
}
}

```

Kode Program 2.18 Program *QuickSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Quick Sort*. Algoritma ini bekerja dengan cara memilih sebuah elemen sebagai *pivot*, kemudian membagi data menjadi dua bagian, yaitu elemen yang lebih kecil dari pivot dan elemen yang lebih besar dari *pivot*. Setelah itu, proses yang sama dilakukan secara berulang pada kedua bagian tersebut hingga seluruh data terurut.

Pada program ini, pemilihan *pivot* dilakukan menggunakan metode *Median-of-Three*, yaitu dengan mengambil nilai tengah dari tiga elemen (awal, tengah, dan akhir). Teknik ini digunakan untuk menghasilkan

pembagian data yang lebih seimbang sehingga kinerja algoritma menjadi lebih baik.

Method quicksort_2025() digunakan untuk melakukan proses pengurutan secara rekursif, sedangkan *method partition_2025()* digunakan untuk membagi *array* berdasarkan nilai *pivot*.

Pada *method main()*, program membuat *array* berisi data 10, 7, 8, 9, 1, dan 5. Selanjutnya program menampilkan data sebelum diurutkan, memanggil *method quicksort_2025()*, dan menampilkan data setelah proses pengurutan selesai.

2.2.1 Langkah Kerja Program *QuickSort*

1. Membuat *class QuickSort_2511532025*

```
public class QuickSort_2511532025 {
```

Kode Program 2.19 *Class QuickSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Quick Sort*.

2. Membuat *method swap_2025()*

```
static void swap_2025(int[] arr_2025, int i_2025,
int j_2025) {
    int temp_2025 = arr_2025[i_2025];
    arr_2025[i_2025] = arr_2025[j_2025];
    arr_2025[j_2025] = temp_2025;
}
```

Kode Program 2.20 *Method Swap*

Method ini digunakan untuk menukar posisi dua elemen dalam *array*.

3. Membuat *method medianOfThree_2025()*

```
// Metode tambahan untuk mengatur pivot_2025
menggunakan Median-of-Three
static void medianOfThree_2025(int[] arr_2025, int
low_2025, int high_2025) {
    int mid_2025 = low_2025 + (high_2025 -
low_2025)/2;
```

```

        // Urutkan elemen low, mid dan high
        if (arr_2025[low_2025] > arr_2025[mid_2025])
        {
            swap_2025(arr_2025, low_2025,
mid_2025);
        }
        if (arr_2025[low_2025] > arr_2025[high_2025])
        {
            swap_2025(arr_2025, low_2025,
high_2025);
        }
        if (arr_2025[mid_2025] > arr_2025[high_2025])
        {
            swap_2025(arr_2025, mid_2025,
high_2025);
        }
        swap_2025(arr_2025, mid_2025, high_2025);
    }
}

```

Kode Program 2.21 *Method Median Of Three*

Method ini digunakan untuk menentukan *pivot* menggunakan teknik *Median-of-Three* dengan membandingkan elemen awal, tengah, dan akhir *array*.

4. Membuat *method partition_2025()*

```

static int partition_2025(int[] arr_2025, int
low_2025, int high_2025) {

    // Panggil fungsi medianOfThree sebelum
menentukan pivot_2025

```

Kode Program 2.22 *Method Partition*

Method ini digunakan untuk membagi *array* berdasarkan nilai *pivot* yang telah ditentukan.

5. Menentukan *pivot* dan indeks awal

```

    medianOfThree_2025(arr_2025, low_2025,
high_2025);

    int pivot_2025 = arr_2025[high_2025]; //
Sekarang arr_2025[high] sudah berisi nilai median
    int i_2025 = (low_2025- 1);

```

Kode Program 2.23 Menentukan *Pivot*

Syntax ini digunakan untuk menentukan *pivot* dan menyiapkan indeks awal proses *partition*.

6. Membandingkan elemen dengan *pivot*

```
for (int j_2025 = low_2025; j_2025 <=
high_2025 - 1; j_2025++) {
    // Jika elemen saat ini lebih kecil dari
    atau sama dengan pivot_2025
    if (arr_2025[j_2025] < pivot_2025) {
        // Increment indeks elemen yang
        lebih kecil
        i_2025++;
    }
}
```

Kode Program 2.24 Proses Perbandingan

Perulangan digunakan untuk membandingkan setiap elemen dengan *pivot*. Jika elemen lebih kecil dari *pivot*, maka elemen dipindahkan ke bagian kiri *array*.

7. Menempatkan *pivot* pada posisi yang benar

```
swap_2025(arr_2025, i_2025,
j_2025);
    }
}
swap_2025(arr_2025, i_2025 + 1, high_2025);
return (i_2025 + 1);
}
```

Kode Program 2.25 Menempatkan *Pivot*

Syntax ini digunakan untuk menempatkan *pivot* pada posisi yang sesuai dan mengembalikan indeks *pivot*.

8. Membuat *method quicksort_2025()*

```
static void quicksort_2025(int[] arr_2025, int
low_2025, int high_2025) {
```

Kode Program 2.26 *Method QuickSort*

Method ini digunakan untuk mengurutkan *array* secara rekursif.

9. Melakukan proses rekursif

```
        if (low_2025 < high_2025) {
            int pi_2025 = partition_2025(arr_2025,
low_2025, high_2025);
            quicksort_2025(arr_2025, low_2025,
pi_2025 - 1);
            quicksort_2025(arr_2025, pi_2025 + 1,
high_2025);
        }
    }
```

Kode Program 2.27 Rekursi *QuickSort*

Syntax ini digunakan untuk mengurutkan bagian kiri dan kanan *pivot* hingga seluruh data terurut.

10. Membuat *method printArr_2025()* dan *method main()*

```
public static void printArr_2025(int[] arr_2025) {
    for(int i_2025 = 0; i_2025 < arr_2025.length;
i_2025++) {
        System.out.print(arr_2025[i_2025] + "
");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_2025 = { 10, 7, 8, 9, 1, 5 };
    int N_2025 = arr_2025.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2025(arr_2025);

    quicksort_2025(arr_2025, 0, N_2025 - 1);

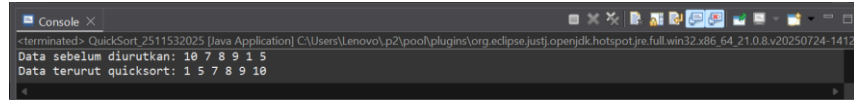
    System.out.print("Data terurut quicksort: ");
    printArr_2025(arr_2025);
}
}
```

Kode Program 2.28 *Method Output* dan *Main*

Method printArr_2025() digunakan untuk menampilkan isi *array*, sedangkan *method main()* digunakan untuk membuat

data, menjalankan proses pengurutan, dan menampilkan hasil sebelum serta sesudah *sorting*.

2.2.2 Hasil Program *QuickSort*



Gambar 2.2 Hasil Program *QuickSort*

Berdasarkan hasil eksekusi program, algoritma *Quick Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi *array* adalah:

10 7 8 9 1 5

Setelah *method quicksort_2025()* dijalankan, *array* berhasil diurutkan menjadi:

1 5 7 8 9 10

Hasil tersebut menunjukkan bahwa algoritma *Quick Sort* bekerja dengan cara memilih sebuah *pivot*, kemudian membagi data menjadi dua bagian berdasarkan nilai *pivot* tersebut. Proses pembagian dan pengurutan dilakukan secara rekursif hingga seluruh elemen *array* tersusun secara terurut dari nilai terkecil ke terbesar.

2.3 Program *MergeSort*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan8_2511532025;

public class MergeSort_2511532025 {
    void merge_2025(int arr_2025[], int l_2025, int m_2025, int r_2025) {
        // Find sizes of two subarrays to be merged
        int n1_2025 = m_2025 - l_2025 + 1;
        int n2_2025 = r_2025 - m_2025;
        /* Create temp arrays */
        int L_2025[] = new int[n1_2025];
        int R_2025[] = new int[n2_2025];
        /* Copy data to temp arrays */
        for (int i_2025 = 0; i_2025 < n1_2025; ++i_2025)
            L_2025[i_2025] = arr_2025[l_2025 + i_2025];
    }
}
```

```

        for (int j_2025 = 0; j_2025 < n2_2025; ++j_2025)
            R_2025[j_2025] = arr_2025[m_2025 + 1 +
j_2025];
        int i_2025 = 0, j_2025 = 0;
        // Initial index of merged subarray array
        int k_2025 = l_2025;
        while (i_2025 < n1_2025 && j_2025 < n2_2025) {
            if (L_2025[i_2025] <= R_2025[j_2025]) {
                arr_2025[k_2025] = L_2025[i_2025];
                i_2025++;
            } else {
                arr_2025[k_2025] = R_2025[j_2025];
                j_2025++;
            }
            k_2025++;
        }
        /* Copy remaining elements of L[] if any */
        while (i_2025 < n1_2025) {
            arr_2025[k_2025] = L_2025[i_2025];
            i_2025++;
            k_2025++;
        }
        /* Copy remaining elements of R[] any */
        while (j_2025 < n2_2025) {
            arr_2025[k_2025] = R_2025[j_2025];
            j_2025++;
            k_2025++;
        }
    }

    void sort_2025(int arr_2025[], int l_2025, int r_2025) {
        if (l_2025 < r_2025) {
            // Find the middle point
            int m_2025 = (l_2025 + r_2025)/2;
            // Sort first and second halves
            sort_2025 (arr_2025, l_2025, m_2025);
            sort_2025 (arr_2025, m_2025 + 1, r_2025);
            merge_2025(arr_2025, l_2025, m_2025, r_2025);
        }
    }

    static void printArray_2025(int arr_2025[]) {
        int n_2025 = arr_2025.length;
        for (int i_2025 = 0; i_2025 < n_2025;
++i_2025)
            System.out.print(arr_2025[i_2025] + "
");
        System.out.println();
    }

    public static void main(String[] args) {
        int arr_2025[] = { 12, 11, 13, 5, 6, 7 };
        System.out.println("Sebelum terurut");
        printArray_2025(arr_2025);
        MergeSort_2511532025 ob_2025 = new
MergeSort_2511532025();
        ob_2025.sort_2025(arr_2025, 0, arr_2025.length - 1);
        System.out.println("\nSesudah Terurut menggunakan
merge sort");
        printArray_2025(arr_2025);
    }
}

```

```
}
}
```

Kode Program 2.29 Program *MergeSort*

Kode program di atas digunakan untuk mengurutkan data menggunakan algoritma *Merge Sort*. Algoritma ini bekerja dengan cara membagi *array* menjadi beberapa bagian yang lebih kecil hingga setiap bagian hanya terdiri dari satu elemen. Setelah itu, bagian-bagian tersebut digabungkan kembali secara berurutan sehingga menghasilkan *array* yang terurut.

Method sort_2025() digunakan untuk membagi *array* secara rekursif menjadi dua bagian yang lebih kecil. Setelah proses pembagian selesai, *method merge_2025()* digunakan untuk menggabungkan kembali kedua bagian *array* tersebut dengan membandingkan setiap elemen sehingga diperoleh urutan data yang benar.

Pada program ini, *method merge_2025()* bertugas menggabungkan dua *sub-array* yang telah terurut menjadi satu *array* yang terurut. Sedangkan *method printArray_2025()* digunakan untuk menampilkan isi *array* sebelum dan sesudah proses pengurutan.

Pada *method main()*, program membuat *array* berisi data 12, 11, 13, 5, 6, dan 7. Selanjutnya program menampilkan data sebelum diurutkan, membuat objek *MergeSort_2511532025*, kemudian memanggil *method sort_2025()* untuk melakukan pengurutan data. Setelah proses selesai, program menampilkan hasil *array* yang telah terurut.

2.3.1 Langkah Kerja Program *MergeSort*

1. Membuat *Class MergeSort_2511532025*

```
public class MergeSort_2511532025 {
```

Kode Program 2.30 *Class MergeSort*

Class ini digunakan untuk menjalankan proses pengurutan data menggunakan algoritma *Merge Sort*.

2. Membuat *method merge_2025*

```
void merge_2025(int arr_2025[], int l_2025, int m_2025, int r_2025) {
```

Kode Program 2.31 *Method Merge*

Method ini digunakan untuk menggabungkan dua *sub-array* yang telah terurut menjadi satu *array* yang terurut.

3. Menentukan ukuran *sub-array* dan membuat *array* sementara

```
// Find sizes of two subarrays to be merged
int n1_2025 = m_2025 - l_2025 + 1;
int n2_2025 = r_2025 - m_2025;
/* Create temp arrays */
int L_2025[] = new int[n1_2025];
int R_2025[] = new int[n2_2025];
```

Kode Program 2.32 Membuat *Sub-array*

Syntax ini digunakan untuk menentukan ukuran *sub-array* kiri dan kanan serta membuat *array* sementara yang akan digunakan dalam proses penggabungan.

4. Menyalin data ke *array* sementara

```
/* Copy data to temp arrays */
for (int i_2025 = 0; i_2025 < n1_2025; ++i_2025)
    L_2025[i_2025] = arr_2025[l_2025 + i_2025];
for (int j_2025 = 0; j_2025 < n2_2025; ++j_2025)
    R_2025[j_2025] = arr_2025[m_2025 + 1 + j_2025];
int i_2025 = 0, j_2025 = 0;
```

Kode Program 2.33 Menyalin Data

Syntax ini digunakan untuk menyalin elemen *array* ke dalam *sub-array* kiri dan kanan.

5. Membandingkan dan menggabungkan *sub-array*

```
// Initial index of merged subarray array
int k_2025 = l_2025;
while (i_2025 < n1_2025 && j_2025 < n2_2025)
{
    if (L_2025[i_2025] <= R_2025[j_2025]) {
        arr_2025[k_2025] =
L_2025[i_2025];
        i_2025++;
    } else {
        arr_2025[k_2025] =
R_2025[j_2025];
        j_2025++;
    }
    k_2025++;
}
```

Kode Program 2.34 Proses Penggabungan

Perulangan ini digunakan untuk membandingkan elemen pada *sub-array* kiri dan kanan, kemudian menempatkan elemen yang lebih kecil ke *array* utama.

6. Menyalin sisa elemen yang belum di proses

```
/* Copy remaining elements of L[] if any */
while (i_2025 < n1_2025) {
    arr_2025[k_2025] = L_2025[i_2025];
    i_2025++;
    k_2025++;
}
/* Copy remaining elements of R[] any */
while (j_2025 < n2_2025) {
    arr_2025[k_2025] = R_2025[j_2025];
    j_2025++;
    k_2025++;
}
}
```

Kode Program 2.35 Menyalin sisa elemen

Syntax ini digunakan untuk menyalin sisa elemen dari *sub-array* kiri atau kanan yang belum dipindahkan ke *array* utama.

7. Membuat *method sort_2025()*

```
void sort_2025(int arr_2025[], int l_2025, int  
r_2025) {  
    if (l_2025 < r_2025) {
```

Kode Program 2.36 *Method Sort*

Method ini digunakan untuk membagi *array* menjadi bagian yang lebih kecil dan menjalankan proses *Merge Sort* secara rekursif.

8. Menentukan titik Tengah dan membagi *array*

```
// Find the middle point  
int m_2025 = (l_2025 + r_2025)/2;  
// Sort first and second halves  
sort_2025 (arr_2025, l_2025, m_2025);  
sort_2025 (arr_2025, m_2025 + 1,  
r_2025);
```

Kode Program 2.37 Membagi *Array*

Syntax ini digunakan untuk mencari titik tengah *array* dan membagi *array* menjadi dua bagian yang lebih kecil.

9. Menggabungkan kembali *array* yang telah diurutkan

```
merge_2025(arr_2025, l_2025, m_2025,  
r_2025);  
}  
}
```

Kode Program 2.38 Menggabungkan *Array*

Syntax ini digunakan untuk menggabungkan dua bagian *array* yang telah terurut menjadi satu *array* yang terurut.

10. Membuat *method printArray_2025()*

```
static void printArray_2025(int arr_2025[]) {  
    int n_2025 = arr_2025.length;  
    for (int i_2025 = 0; i_2025 < n_2025;  
++i_2025)
```

```

System.out.print(arr_2025[i_2025] + " ");
                System.out.println();
        }
    }
}

```

Kode Program 2.39 *Method Print Array*

Method ini digunakan untuk menampilkan isi *array* ke layar.

11. Membuat *method main()*

```

public static void main(String[] args) {

```

Kode Program 2.40 *Method Main*

Method utama yang digunakan untuk menjalankan program *Merge Sort*.

12. Membuat data dan menjalankan proses pengurutan

```

int arr_2025[] = { 12, 11, 13, 5, 6, 7 };

```

```

MergeSort_2511532025 ob_2025 = new
MergeSort_2511532025();
    ob_2025.sort_2025(arr_2025, 0,
arr_2025.length - 1);

```

Kode Program 2.41 Menjalankan *Merge Sort*

Array digunakan sebagai data yang akan diurutkan, kemudian *method sort_2025()* dipanggil untuk menjalankan proses pengurutan.

13. Menampilkan hasil pengurutan

```

System.out.println("Sebelum terurut");
printArray_2025(arr_2025);

```

```

System.out.println("\nSesudah Terurut
menggunakan merge sort");
printArray_2025(arr_2025);

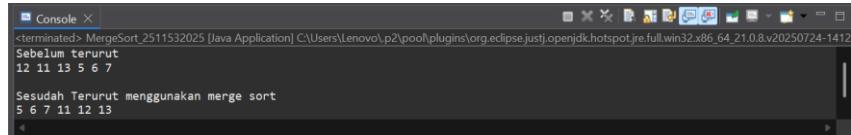
```

```
}
```

Kode Program 2.42 Menampilkan Hasil *Sorting*

Syntax ini digunakan untuk menampilkan data sebelum dan sesudah proses pengurutan dilakukan.

2.3.2 Hasil Program *MergeSort*



Gambar 2.3 Hasil Program *MergeSort*

Berdasarkan hasil eksekusi program, algoritma *Merge Sort* berhasil mengurutkan data dari urutan acak menjadi urutan menaik. Sebelum dilakukan proses pengurutan, isi array adalah:

12 11 13 5 6 7

Setelah *method sort_2025()* dijalankan, *array* berhasil diurutkan menjadi:

5 6 7 11 12 13

Hasil tersebut menunjukkan bahwa algoritma *Merge Sort* bekerja dengan cara membagi *array* menjadi beberapa bagian yang lebih kecil, kemudian mengurutkan setiap bagian secara rekursif dan menggabungkannya kembali hingga seluruh elemen tersusun secara terurut dari nilai terkecil ke terbesar.

2.4 Program *BubbleSortGUI*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan8_2511532025;  
  
import java.awt.BorderLayout;  
import java.awt.Color;  
import java.awt.Dimension;  
import java.awt.EventQueue;
```

```

import java.awt.FlowLayout;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511532025 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2025;
    private JLabel[] labelArray_2025;
    private JButton stepButton_2025, resetButton_2025,
setButton_2025;
    private JTextField inputField_2025;
    private JPanel panelArray_2025;
    private JTextArea stepArea_2025;

    private int i_2025 = 1, j_2025;
    private boolean sorting_2025 = false;
    private int stepCount_2025 = 1;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511532025() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
        inputField_2025 = new JTextField(30);
        setButton_2025 = new JButton("Set Array");
        inputPanel_2025.add(new JLabel("Masukkan angka
(pisahkan dengan koma):"));
        inputPanel_2025.add(inputField_2025);
        inputPanel_2025.add(setButton_2025);

        // Panel array visual
        panelArray_2025 = new JPanel();
        panelArray_2025.setLayout(new FlowLayout());

        // Panel kontrol

```

```

        JPanel controlPanel_2025 = new JPanel();
        stepButton_2025 = new JButton("Langkah
Selanjutnya");
        resetButton_2025 = new JButton("Reset");
        stepButton_2025.setEnabled(false);
        controlPanel_2025.add(stepButton_2025);
        controlPanel_2025.add(resetButton_2025);

        // Area teks untuk log langkah-langkah
        stepArea_2025 = new JTextArea(8, 60);
        stepArea_2025.setEditable(false);
        stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
        JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025);

        // Tambahkan panel ke frame
        add(inputPanel_2025, BorderLayout.NORTH);
        add(panelArray_2025, BorderLayout.CENTER);
        add(controlPanel_2025, BorderLayout.SOUTH);
        add(scrollPane_2025, BorderLayout.EAST);

        // Event Set Array
        setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

        // Event Langkah Selanjutnya
        stepButton_2025.addActionListener(e ->
performStep_2025());

        // Event Reset
        resetButton_2025.addActionListener(e ->
reset_2025());
    }

    private void setArrayFromInput_2025() {
        String text_2025 = inputField_2025.getText().trim();
        if (text_2025.isEmpty()) return;
        String[] parts_2025 = text_2025.split(",");
        array_2025 = new int[parts_2025.length];
        try {
            for(int k_2025 = 0; k_2025 <
parts_2025.length; k_2025++) {
                array_2025[k_2025] =
Integer.parseInt(parts_2025[k_2025].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka " + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2025 = 0;
        j_2025 = 0;
        stepCount_2025 = 1;
        sorting_2025 = true;
    }

```

```

        stepButton_2025.setEnabled(true);
        stepArea_2025.setText("");
        panelArray_2025.removeAll();
        labelArray_2025 = new JLabel[array_2025.length];
        for (int k_2025 = 0; k_2025 < array_2025.length;
k_2025++) {
            labelArray_2025[k_2025] = new
JLabel(String.valueOf(array_2025[k_2025]));
            labelArray_2025[k_2025].setFont (new Font
("Arial", Font.BOLD, 24));
            labelArray_2025[k_2025].setOpaque(true);

            labelArray_2025[k_2025].setBackground(Color.WHITE);

            labelArray_2025[k_2025].setBorder(BorderFactory.createLineB
order(Color.BLACK));
            labelArray_2025[k_2025].setPreferredSize (new
Dimension(50, 50));

            labelArray_2025[k_2025].setHorizontalAlignment(SwingConstan
ts.CENTER);
            panelArray_2025.add
(labelArray_2025[k_2025]);
        }
        panelArray_2025.revalidate();
        panelArray_2025.repaint();
    }
    private void performStep_2025() {
        if (!sorting_2025 || i_2025 >=
array_2025.length - 1) {
            sorting_2025 = false;
            stepButton_2025.setEnabled(false);
            JOptionPane.showMessageDialog(this,
"Sorting selesai!");
            return;
        }
        resetHighlights_2025();
        StringBuilder stepLog = new StringBuilder();

        labelArray_2025[j_2025].setBackground(Color.CYAN);
        labelArray_2025[j_2025 +
1].setBackground(Color.CYAN);
        if (array_2025[j_2025] > array_2025[j_2025 +
1]) {
            // Swap
            int temp_2025 = array_2025[j_2025];
            array_2025[j_2025] = array_2025[j_2025 + 1];
            array_2025[j_2025 + 1] = temp_2025;

            labelArray_2025[j_2025].setBackground(Color.RED);
            labelArray_2025[j_2025 +
1].setBackground(Color.RED);
            stepLog.append("Langkah
").append(stepCount_2025).append(": Menukar elemen ke-
").append(j_2025).append(" (").append(array_2025[j_2025 +
1]).append(") dengan ke-").append(j_2025 + 1).append(" (
").append(array_2025[j_2025]).append(")\n");

```

```

        } else {
            stepLog.append("Langkah
").append(stepCount_2025).append(": Tidak ada pertukaran
antara ke-").append(j_2025).append(" dan ke-").append(j_2025
+ 1).append("\n");
        }
        stepLog.append("Hasil:
").append(arrayToString_2025(array_2025)).append("\n\n");
        stepArea_2025.append(stepLog.toString());
        updateLabels_2025();
        j_2025++;
        if (j_2025 >= array_2025.length - i_2025 - 1) {
            j_2025 = 0;
            i_2025++;
        }
        stepCount_2025++;
        if (i_2025 >= array_2025.length - 1) {
            sorting_2025 = false;
            stepButton_2025.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }
    }
    private void updateLabels_2025() {
        for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {

            labelArray_2025[k_2025].setText(String.valueOf(array_2025[k
_2025]));
        }
    }
    private void resetHighlights_2025() {
        for (JLabel label_2025 : labelArray_2025) {
            label_2025.setBackground(Color.WHITE);
        }
    }
    private void reset_2025() {
        inputField_2025.setText("");
        panelArray_2025.removeAll();
        panelArray_2025.revalidate();
        panelArray_2025.repaint();
        stepArea_2025.setText("");
        stepButton_2025.setEnabled(false);
        sorting_2025 = false;
        i_2025 = 0;
        j_2025 = 0;
        stepCount_2025 = 1;
    }
    private String arrayToString_2025(int[] arr) {
        StringBuilder sb_2025 = new StringBuilder();
        for (int k_2025 = 0; k_2025 < arr.length;
k_2025++) {
            sb_2025.append(arr[k_2025]);
            if (k_2025 < arr.length - 1)
sb_2025.append(", ");
        }
        return sb_2025.toString();
    }

```

```

    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubbleSortGUI_2511532025 gui = new
BubbleSortGUI_2511532025();
            gui.setVisible(true);
        });
    }
}

```

Kode Program 2.43 Program *BubbleSortGUI*

Kode program di atas digunakan untuk memvisualisasikan proses pengurutan data menggunakan algoritma *Bubble Sort* secara bertahap. Program dibuat menggunakan *Java Swing* sehingga pengguna dapat memasukkan data, melihat proses perbandingan dan pertukaran elemen, serta mengikuti setiap langkah pengurutan melalui antarmuka grafis.

Pada program ini, pengguna dapat memasukkan sejumlah angka yang dipisahkan dengan tanda koma. Setelah tombol *Set Array* ditekan, data akan ditampilkan dalam bentuk label. Tombol Langkah Selanjutnya digunakan untuk menjalankan satu langkah proses *Bubble Sort*, sedangkan tombol *Reset* digunakan untuk mengembalikan program ke kondisi awal.

Method performStep_2025() merupakan *method* utama yang menjalankan proses *Bubble Sort* secara bertahap. Setiap langkah perbandingan dan pertukaran elemen akan ditampilkan pada area log sehingga pengguna dapat memahami cara kerja algoritma *Bubble Sort*.

2.4.1 Langkah Kerja Program *BubbleSortGUI*

1. Membuat *class BubbleSortGUI_2511532025*

```
public class BubbleSortGUI_2511532025 extends JFrame {
```

Kode Program 2.44 *Class BubbleSortGUI*

Class ini digunakan untuk membuat aplikasi *GUI* yang memvisualisasikan proses pengurutan menggunakan algoritma *Bubble Sort*.

2. Mendeklarasikan variabel dan komponen *GUI*

```
private static final long serialVersionUID = 1L;
private int[] array_2025;
private JLabel[] labelArray_2025;
private JButton stepButton_2025, resetButton_2025,
setButton_2025;
private JTextField inputField_2025;
private JPanel panelArray_2025;
private JTextArea stepArea_2025;

private int i_2025 = 1, j_2025;
private boolean sorting_2025 = false;
private int stepCount_2025 = 1;
```

Kode Program 2.45 Deklarasi Variabel

Variabel dan komponen *GUI* digunakan untuk menyimpan data *array*, menampilkan elemen *array*, menerima input pengguna, serta mengontrol jalannya program.

3. Membuat tampilan *GUI*

```
public BubbleSortGUI_2511532025() {
    setTitle("Bubble Sort Langkah per Langkah");
    setSize(750, 400);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
}
```

Kode Program 2.46 *Constructor GUI*

Constructor digunakan untuk mengatur ukuran *frame*, *layout*, *panel*, tombol, area input, dan area log yang digunakan dalam aplikasi.

4. Membuat panel input data

```
// Panel input
```

```

        JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
        inputField_2025 = new JTextField(30);
        setButton_2025 = new JButton("Set Array");
        inputPanel_2025.add(new JLabel("Masukkan
angka (pisahkan dengan koma):"));
        inputPanel_2025.add(inputField_2025);
        inputPanel_2025.add(setButton_2025);

```

Kode Program 2.47 Panel Input

Komponen ini digunakan untuk menerima data angka dari pengguna yang dipisahkan dengan tanda koma.

5. Membuat panel visualisasi *array*

```

// Panel array visual
panelArray_2025 = new JPanel();
panelArray_2025 .setLayout(new FlowLayout());

```

Kode Program 2.48 Panel *Array*

Panel ini digunakan untuk menampilkan elemen-elemen *array* dalam bentuk label sehingga proses pengurutan dapat diamati secara visual.

6. Membuat tombol kontrol

```

// Panel kontrol
JPanel controlPanel_2025 = new JPanel();
stepButton_2025 = new JButton("Langkah
Selanjutnya");
resetButton_2025 = new JButton("Reset");
stepButton_2025 .setEnabled(false);
controlPanel_2025 .add(stepButton_2025 );
controlPanel_2025 .add(resetButton_2025 );

```

Kode Program 2.49 Tombol Kontrol

Tombol digunakan untuk menjalankan langkah pengurutan berikutnya dan mengembalikan program ke kondisi awal.

7. Membuat area *log* langkah pengurutan

```
// Area teks untuk log langkah-langkah
stepArea_2025 = new JTextArea(8, 60);
stepArea_2025.setEditable(false);
stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025 );

// Tambahkan panel ke frame
add(inputPanel_2025 , BorderLayout.NORTH);
add(panelArray_2025 , BorderLayout.CENTER);
add(controlPanel_2025 , BorderLayout.SOUTH);
add(scrollPane_2025 , BorderLayout.EAST);
```

Kode Program 2.50 Area *Log*

Area teks digunakan untuk menampilkan informasi setiap langkah proses *Bubble Sort* yang sedang berlangsung.

8. Membuat *event* tombol *Set Array*

```
// Event Set Array
setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

// Event Langkah Selanjutnya
stepButton_2025.addActionListener(e ->
performStep_2025());

// Event Reset
resetButton_2025.addActionListener(e ->
reset_2025());

}
```

Kode Program 2.51 *Event Set Array*

Event ini digunakan untuk membaca input pengguna dan mengubahnya menjadi *array* yang siap diurutkan.

9. Membuat *method* `setArrayFromInput_2025()`

```
private void setArrayFromInput_2025() {
    String text_2025 =
inputField_2025.getText().trim();
    if (text_2025.isEmpty()) return;
    String[] parts_2025 = text_2025.split(",");
    array_2025 = new int[parts_2025.length];
    try {
        for(int k_2025 = 0; k_2025 <
parts_2025.length; k_2025++) {
            array_2025[k_2025] =
Integer.parseInt(parts_2025[k_2025].trim());
        }
    } catch (NumberFormatException e) {

        JOptionPane.showMessageDialog(this, "Masukkan hanya
angka " + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_2025 = 0;
    j_2025 = 0;
    stepCount_2025 = 1;
    sorting_2025 = true;
    stepButton_2025.setEnabled(true);
    stepArea_2025.setText("");
    panelArray_2025.removeAll();
    labelArray_2025 = new
JLabel[array_2025.length];
    for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {
        labelArray_2025[k_2025] = new
JLabel(String.valueOf(array_2025[k_2025]));
        labelArray_2025[k_2025].setFont (new
Font ("Arial", Font.BOLD, 24));

        labelArray_2025[k_2025].setOpaque(true);

        labelArray_2025[k_2025].setBackground(Color.WHITE);

        labelArray_2025[k_2025].setBorder(BorderFactory.crea
teLineBorder(Color.BLACK));

        labelArray_2025[k_2025].setPreferredSize (new
Dimension(50, 50));

        labelArray_2025[k_2025].setHorizontalAlignment(Swing
Constants.CENTER);
        panelArray_2025.add
(labelArray_2025[k_2025]);
    }
}
```

```

panelArray_2025.revalidate();
panelArray_2025.repaint();
}

```

Kode Program 2.52 *Method Set Array*

Method ini digunakan untuk mengambil input angka dari pengguna, mengubahnya menjadi *array* integer, dan menampilkan *array* tersebut pada panel visualisasi.

10. Membuat *method performStep_2025()*

```

private void performStep_2025() {
    if (!sorting_2025 || i_2025 >=
array_2025.length - 1) {
        sorting_2025 = false;

        stepButton_2025.setEnabled(false);

        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        return;
    }
    resetHighlights_2025();
    StringBuilder stepLog = new
StringBuilder();

    labelArray_2025[j_2025].setBackground(Color.CYAN);
    labelArray_2025[j_2025 +
1].setBackground(Color.CYAN);
}

```

Kode Program 2.53 *Method Perform Step*

Method ini merupakan inti program yang menjalankan satu langkah *Bubble Sort* setiap kali tombol “Langkah Selanjutnya” ditekan.

11. Membandingkan dan menukar elemen

```

if (array_2025[j_2025] >
array_2025[j_2025 + 1]) {
    // Swap
    int temp_2025 = array_2025[j_2025];
    array_2025[j_2025] =
array_2025[j_2025 + 1];
    array_2025[j_2025 + 1] = temp_2025;
    labelArray_2025[j_2025].setBackground(Color.RED);
}

```

```

        labelArray_2025[j_2025 +
1].setBackground(Color.RED);
        stepLog.append("Langkah
").append(stepCount_2025).append(": Menukar elemen ke-
").append(j_2025).append("
").append(array_2025[j_2025 + 1]).append(" dengan
ke-").append(j_2025 + 1).append("
").append(array_2025[j_2025]).append(")\n");
    } else {
        stepLog.append("Langkah
").append(stepCount_2025).append(": Tidak ada
pertukaran antara ke-").append(j_2025).append(" dan
ke-").append(j_2025 + 1).append(")\n");
    }
    stepLog.append("Hasil:
").append(arrayToString_2025(array_2025)).append("\n\n
");
    stepArea_2025.append(stepLog.toString());
    updateLabels_2025();
    j_2025++;
    if (j_2025 >= array_2025.length - i_2025
- 1) {
        j_2025 = 0;
        i_2025++;
    }
    stepCount_2025++;
    if (i_2025 >= array_2025.length - 1) {
        sorting_2025 = false;
        stepButton_2025.setEnabled(false);
        JOptionPane.showMessageDialog(this,
"Sorting selesai!");
    }
}

```

Kode Program 2.54 Proses *Bubble Sort*

Syntax ini digunakan untuk membandingkan dua elemen yang berdekatan. Jika elemen kiri lebih besar daripada elemen kanan, maka kedua elemen akan ditukar.

12. Memperbarui tampilan *array*

```

// Area teks untuk log langkah-langkah
stepArea_2025 = new JTextArea(8, 60);
stepArea_2025.setEditable(false);
stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025 );

// Tambahkan panel ke frame
add(inputPanel_2025 , BorderLayout.NORTH);

```

```
add(panelArray_2025 , BorderLayout.CENTER);
add(controlPanel_2025 , BorderLayout.SOUTH);
add(scrollPane_2025 , BorderLayout.EAST);
```

Kode Program 2.55 *Update Tampilan*

Method ini digunakan untuk memperbarui nilai label sehingga perubahan hasil pengurutan dapat langsung terlihat pada layar.

13. Membuat *method reset_2025()*

```
private void updateLabels_2025() {
    for (int k_2025 = 0; k_2025 <
array_2025.length; k_2025++) {

        labelArray_2025[k_2025].setText(String.valueOf(array
_2025[k_2025]));
    }

    private void resetHighlights_2025() {
        for (JLabel label_2025 :
labelArray_2025) {

            label_2025.setBackground(Color.WHITE);
        }
    }
}
```

Kode Program 2.56 *Method Reset*

Method ini digunakan untuk menghapus data, membersihkan tampilan, dan mengembalikan seluruh variabel ke kondisi awal.

14. Membuat *method arrayToString_2025()*

```
private void reset_2025() {
    inputField_2025.setText("");
    panelArray_2025.removeAll();
    panelArray_2025.revalidate();
    panelArray_2025.repaint();
    stepArea_2025.setText("");
    stepButton_2025.setEnabled(false);
    sorting_2025 = false;
    i_2025 = 0;
    j_2025 = 0;
    stepCount_2025 = 1;
}
```

```

        private String arrayToString_2025(int[] arr)
        {
            StringBuilder sb_2025 = new
StringBuilder();
            for (int k_2025 = 0; k_2025 <
arr.length; k_2025++) {
                sb_2025.append(arr[k_2025]);
                if (k_2025 < arr.length - 1)
sb_2025.append(", ");
            }
            return sb_2025.toString();
        }

```

Kode Program 2.57 Konversi *Array*

Method ini digunakan untuk mengubah isi *array* menjadi bentuk teks yang ditampilkan pada area *log*.

15. Membuat *method main()*

```

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> {
                BubbleSortGUI_2511532025 gui =
new BubbleSortGUI_2511532025();
                gui.setVisible(true);
            });
        }
    }

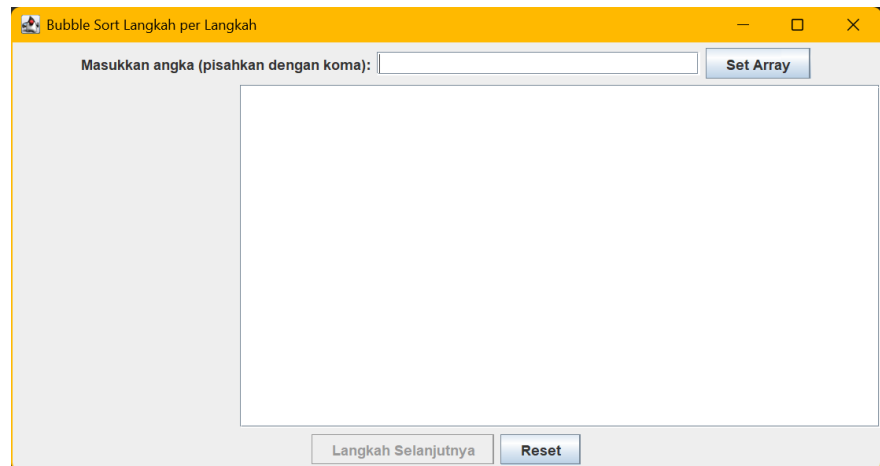
```

Kode Program 2.58 *Method Main*

Method utama digunakan untuk menjalankan aplikasi *GUI Bubble Sort* dan menampilkan jendela program kepada pengguna.

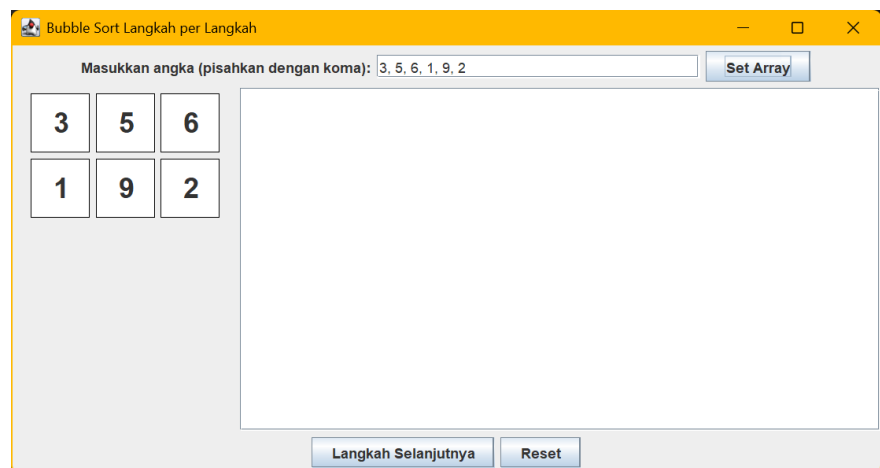
2.4.2 Hasil Program

Tampilan awal program



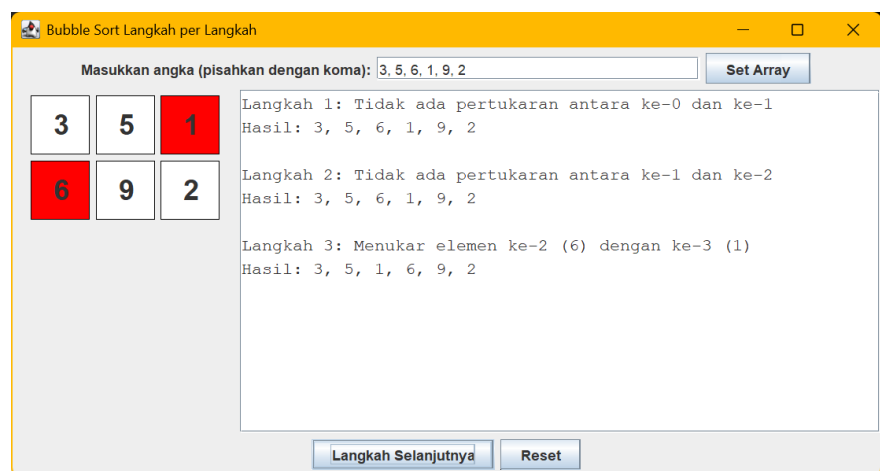
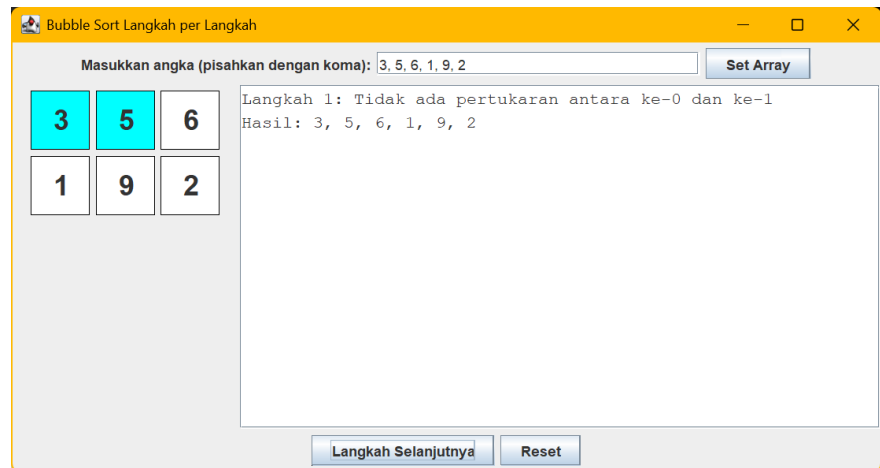
Gambar 2.4 Hasil Program Tampilan Awal Program
BubbleSortGUI

Input data *array*

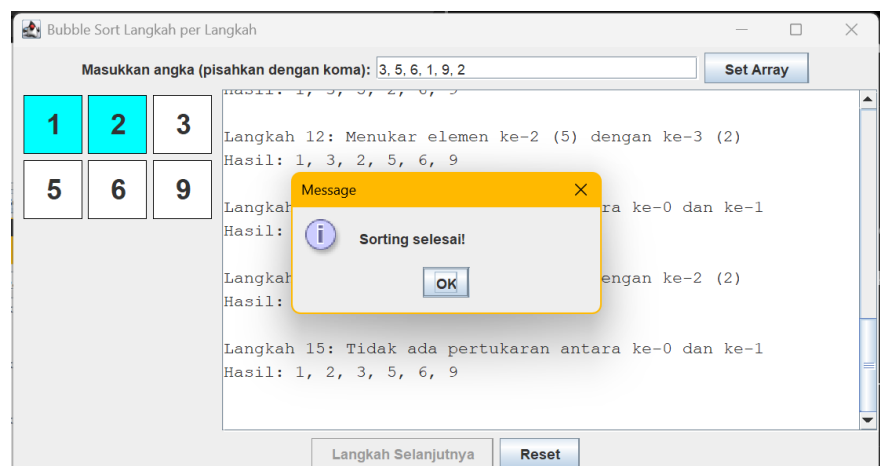


Gambar 2.5 Tampilan Data Sebelum Pengurutan

Proses *Bubble Sort* berlangsung



Gambar 2.6 Hasil Proses Perbandingan dan Pertukaran Data
Hasil akhir *sorting*



Gambar 2.7 Hasil Hasil Pengurutan Data Menggunakan *Bubble Sort*

Berdasarkan hasil eksekusi program, pengguna dapat memasukkan data berupa angka yang dipisahkan dengan tanda koma. Setelah tombol “Set Array” ditekan, data ditampilkan dalam bentuk label pada panel visualisasi. Tombol “Langkah Selanjutnya” digunakan untuk menjalankan proses *Bubble Sort* secara bertahap, sehingga setiap proses perbandingan dan pertukaran elemen dapat diamati dengan jelas.

Setelah seluruh langkah selesai dijalankan, *array* berhasil diurutkan dari nilai terkecil ke terbesar. Program juga menampilkan *log* setiap langkah pengurutan sehingga memudahkan pengguna dalam memahami cara kerja algoritma *Bubble Sort* secara visual.

2.5 Program *MergeSortGUI*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan8_2511532025;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

import java.util.LinkedList;
import java.util.Queue;

public class MergeSortGUI_2511532025 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2025;
    private JLabel[] labelArray_2025;
```

```

        private JButton stepButton_2025, resetButton_2025,
setButton_2025;
        private JTextField inputField_2025;
        private JPanel panelArray_2025;
        private JTextArea stepArea_2025;

        private Queue<int[]> mergeQueue_2025 = new LinkedList<>();
        private int stepCount_2025 = 1;
        private boolean isMerging_2025 = false;
        private boolean copying_2025 = false;

        private int[] temp_2025;
        private int left_2025, mid_2025, right_2025;
        private int i_2025, j_2025, k_2025;

        public MergeSortGUI_2511532025() {
            setTitle("Merge Sort Langkah per Langkah");
            setSize(750, 400);
            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            setLocationRelativeTo(null);
            setLayout(new BorderLayout());

            // panel input
            JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
            inputField_2025 = new JTextField(30);
            setButton_2025 = new JButton("Set Array");
            inputPanel_2025.add(new JLabel("Masukan angka
(pisahkan dengan koma)"));
            inputPanel_2025.add(inputField_2025);
            inputPanel_2025.add(setButton_2025);

            // panel array visual
            panelArray_2025 = new JPanel();
            panelArray_2025.setLayout(new FlowLayout());

            // panel kontrol
            JPanel controlPanel_2025 = new JPanel();
            stepButton_2025 = new JButton("Langkah
selanjutnya");
            resetButton_2025 = new JButton("Reset");
            controlPanel_2025.add(stepButton_2025);
            controlPanel_2025.add(resetButton_2025);

            // area text untuk log langkah langkah
            stepArea_2025 = new JTextArea(8, 60);
            stepArea_2025.setEditable(false);
            stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
            JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025);

            // tambahkan panel ke frame
            add(inputPanel_2025, BorderLayout.NORTH);
            add(panelArray_2025, BorderLayout.CENTER);
            add(controlPanel_2025, BorderLayout.SOUTH);
            add(scrollPane_2025, BorderLayout.EAST);

```

```

        // event set array
        setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

        // event selanjutnya
        stepButton_2025.addActionListener(e ->
performStep_2025());

        // event reset
        resetButton_2025.addActionListener(e ->
reset_2025());
    }

    private void setArrayFromInput_2025() {
        String text_2025 = inputField_2025.getText().trim();
        if (text_2025.isEmpty()) return;
        String[] parts_2025 = text_2025.split(",");
        array_2025 = new int[parts_2025.length];
        try {
            for (int k = 0; k < parts_2025.length; k++) {
                array_2025[k] =
Integer.parseInt(parts_2025[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan
hanya angka dengan koma!",
                "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        labelArray_2025 = new JLabel[array_2025.length];
        panelArray_2025.removeAll();
        for (int k = 0; k < array_2025.length; k++) {
            labelArray_2025[k] = new
JLabel(String.valueOf(array_2025[k]));
            labelArray_2025[k].setFont(new Font("Arial",
Font.BOLD, 24));
            labelArray_2025[k].setOpaque(true);

            labelArray_2025[k].setBackground(Color.WHITE);

            labelArray_2025[k].setBorder(BorderFactory.createLineBorder
(Color.BLACK));
            labelArray_2025[k].setPreferredSize(new
Dimension(50, 50));

            labelArray_2025[k].setHorizontalAlignment(SwingConstants.CE
NTER);

            panelArray_2025.add(labelArray_2025[k]);
        }

        mergeQueue_2025.clear();
        generateMergeSteps_2025(0, array_2025.length - 1);
        stepButton_2025.setEnabled(true);
        stepArea_2025.setText("");
    }

```

```

        stepCount_2025 = 1;
        isMerging_2025 = false;

        panelArray_2025.revalidate();
        panelArray_2025.repaint();
    }

    private void generateMergeSteps_2025(int left, int right) {
        if (left >= right) return;
        int mid = (left + right) / 2;
        generateMergeSteps_2025(left, mid);
        generateMergeSteps_2025(mid + 1, right);
        mergeQueue_2025.add(new int[]{left, mid, right});
    }

    private void performStep_2025() {
        resetHighlights_2025();

        if (!isMerging_2025 && !mergeQueue_2025.isEmpty()) {
            int[] range_2025 = mergeQueue_2025.poll();
            left_2025 = range_2025[0];
            mid_2025 = range_2025[1];
            right_2025 = range_2025[2];
            temp_2025 = new int[right_2025 - left_2025 +
1];

            i_2025 = left_2025;
            j_2025 = mid_2025 + 1;
            k_2025 = 0;
            copying_2025 = false;
            isMerging_2025 = true;
            stepArea_2025.append("Langkah " +
stepCount_2025++ +
                                ": Mulai merge dari " +
left_2025 + " ke " + right_2025 + "\n");
            return;
        }

        if (isMerging_2025 && !copying_2025) {
            if (i_2025 <= mid_2025 && j_2025 <=
right_2025) {

                labelArray_2025[i_2025].setBackground(Color.CYAN);

                labelArray_2025[j_2025].setBackground(Color.CYAN);
                if (array_2025[i_2025] <=
array_2025[j_2025]) {
                    temp_2025[k_2025++] =
array_2025[i_2025++];
                } else {
                    temp_2025[k_2025++] =
array_2025[j_2025++];
                }
                stepArea_2025.append("Langkah " +
stepCount_2025++ + ": Bandingkan dan salin elemen\n");
                return;
            } else if (i_2025 <= mid_2025) {

```

```

        temp_2025[k_2025++] =
array_2025[i_2025++];
        stepArea_2025.append("Langkah " +
stepCount_2025++ + ": Salin sisa kiri\n");
        return;
    } else if (j_2025 <= right_2025) {
        temp_2025[k_2025++] =
array_2025[j_2025++];
        stepArea_2025.append("Langkah " +
stepCount_2025++ + ": Salin sisa kanan\n");
        return;
    } else {
        copying_2025 = true;
        k_2025 = 0;
        return;
    }
}

if (copying_2025 && k_2025 < temp_2025.length) {
    array_2025[left_2025 + k_2025] =
temp_2025[k_2025];
    labelArray_2025[left_2025 +
k_2025].setText(String.valueOf(temp_2025[k_2025]));
    labelArray_2025[left_2025 +
k_2025].setBackground(Color.GREEN);
    k_2025++;
    stepArea_2025.append("Langkah " +
stepCount_2025++ + ": Tempelkan ke array utama\n");
    return;
}

if (copying_2025 && k_2025 == temp_2025.length) {
    isMerging_2025 = false;
    copying_2025 = false;
}

if (mergeQueue_2025.isEmpty() && !isMerging_2025) {
    stepArea_2025.append("Selesai.\n");
    stepButton_2025.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Merge
Sort selesai!");
}

}

private void resetHighlights_2025() {
    if (labelArray_2025 == null) return;
    for (JLabel label : labelArray_2025) {
        label.setBackground(Color.WHITE);
    }
}

private void reset_2025() {
    inputField_2025.setText("");
    panelArray_2025.removeAll();
    panelArray_2025.revalidate();
    panelArray_2025.repaint();
    stepArea_2025.setText("");
}

```

```

        stepButton_2025.setEnabled(false);
        mergeQueue_2025.clear();
        isMerging_2025 = false;
        stepCount_2025 = 1;
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MergeSortGUI_2511532025 gui = new
MergeSortGUI_2511532025();
            gui.setVisible(true);
        });
    }
}

```

Kode Program 2.59 Program *MergeSortGUI*

Kode program di atas digunakan untuk memvisualisasikan proses pengurutan data menggunakan algoritma *Merge Sort* secara bertahap. Program dibuat menggunakan *Java Swing* sehingga pengguna dapat memasukkan data, melihat proses pembagian dan penggabungan *array*, serta mengikuti setiap langkah pengurutan melalui antarmuka grafis.

Pada program ini, pengguna dapat memasukkan sejumlah angka yang dipisahkan dengan tanda koma. Setelah tombol “Set Array” ditekan, data akan ditampilkan dalam bentuk label. Tombol “Langkah Selanjutnya” digunakan untuk menjalankan satu langkah proses *Merge Sort*, sedangkan tombol *Reset* digunakan untuk mengembalikan program ke kondisi awal.

Method performStep_2025() merupakan *method* utama yang menjalankan proses *Merge Sort* secara bertahap. Setiap proses penggabungan (*merge*) dan pemindahan elemen akan ditampilkan pada area *log* sehingga pengguna dapat memahami cara kerja algoritma *Merge Sort* secara visual.

2.5.1 Langkah Kerja Program *MergeSortGUI*

1. Membuat *class MergeSortGUI_2511532025*

```

public class MergeSortGUI_2511532025 extends JFrame {

```

Kode Program 2.60 *Class MergeSortGUI*

Class ini digunakan untuk membuat aplikasi *GUI* yang memvisualisasikan proses pengurutan menggunakan algoritma *Merge Sort*.

2. Mendeklarasikan variabel dan komponen *GUI*

```
private static final long serialVersionUID = 1L;
private int[] array_2025;
private JLabel[] labelArray_2025;
private JButton stepButton_2025, resetButton_2025,
setButton_2025;
private JTextField inputField_2025;
private JPanel panelArray_2025;
private JTextArea stepArea_2025;

private int i_2025 = 1, j_2025;
private boolean sorting_2025 = false;
private int stepCount_2025 = 1;
```

Kode Program 2.61 Deklarasi Variabel

Variabel dan komponen *GUI* digunakan untuk menyimpan data *array*, menampilkan elemen *array*, menerima input pengguna, serta mengontrol jalannya program.

3. Membuat tampilan *GUI*

```
public MergeSortGUI_2511532025() {
    setTitle("Merge Sort Langkah per Langkah");
    setSize(750, 400);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());
}
```

Kode Program 2.62 *Constructor GUI*

Constructor digunakan untuk mengatur ukuran *frame*, panel input, panel visualisasi *array*, tombol kontrol, dan area *log*.

4. Membuat panel input data

```
// panel input
JPanel inputPanel_2025 = new JPanel(new
FlowLayout());
inputField_2025 = new JTextField(30);
setButton_2025 = new JButton("Set Array");
```

```

        inputPanel_2025.add(new JLabel("Masukan angka
        (pisahkan dengan koma)"));
        inputPanel_2025.add(inputField_2025);
        inputPanel_2025.add(setButton_2025);

```

Kode Program 2.63 Panel Input

Komponen ini digunakan untuk menerima data angka dari pengguna yang dipisahkan dengan tanda koma.

5. Membuat panel visualisasi *array*

```

// panel array visual
panelArray_2025 = new JPanel();
panelArray_2025.setLayout(new FlowLayout());

```

Kode Program 2.64 Panel *Array*

Panel ini digunakan untuk menampilkan elemen-elemen *array* dalam bentuk label sehingga proses pengurutan dapat diamati secara visual.

6. Membuat tombol kontrol

```

// panel kontrol
JPanel controlPanel_2025 = new JPanel();
stepButton_2025 = new JButton("Langkah
selanjutnya");
resetButton_2025 = new JButton("Reset");
controlPanel_2025.add(stepButton_2025);
controlPanel_2025.add(resetButton_2025);

// area text untuk log langkah langkah
stepArea_2025 = new JTextArea(8, 60);
stepArea_2025.setEditable(false);
stepArea_2025.setFont(new Font("Monospaced",
Font.PLAIN, 14));
JScrollPane scrollPane_2025 = new
JScrollPane(stepArea_2025);

// tambahkan panel ke frame
add(inputPanel_2025, BorderLayout.NORTH);
add(panelArray_2025, BorderLayout.CENTER);
add(controlPanel_2025, BorderLayout.SOUTH);
add(scrollPane_2025, BorderLayout.EAST);

```

Kode Program 2.65 Tombol Kontrol

Tombol digunakan untuk menjalankan langkah *Merge Sort* berikutnya dan mengembalikan program ke kondisi awal.

7. Membuat *event* tombol *set array*

```
// event set array
setButton_2025.addActionListener(e ->
setArrayFromInput_2025());

// event selanjutnya
stepButton_2025.addActionListener(e ->
performStep_2025());

// event reset
resetButton_2025.addActionListener(e ->
reset_2025());
}
```

Kode Program 2.66 *Event Set Array*

Event ini digunakan untuk membaca input pengguna dan mengubahnya menjadi *array* yang siap diurutkan.

8. Membuat *method* *setArrayFromInput_2025()*

```
private void setArrayFromInput_2025() {
    String text_2025 =
inputField_2025.getText().trim();
    if (text_2025.isEmpty()) return;
    String[] parts_2025 = text_2025.split(",");
    array_2025 = new int[parts_2025.length];
    try {
        for (int k = 0; k < parts_2025.length;
k++) {
            array_2025[k] =
Integer.parseInt(parts_2025[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this,
"Masukkan hanya angka dengan koma!",
"Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_2025 = new
JLabel[array_2025.length];
    panelArray_2025.removeAll();
}
```

```

        for (int k = 0; k < array_2025.length; k++) {
            labelArray_2025[k] = new
JLabel(String.valueOf(array_2025[k]));
            labelArray_2025[k].setFont(new
Font("Arial", Font.BOLD, 24));
            labelArray_2025[k].setOpaque(true);

            labelArray_2025[k].setBackground(Color.WHITE);

            labelArray_2025[k].setBorder(BorderFactory.createLin
eBorder(Color.BLACK));
            labelArray_2025[k].setPreferredSize(new
Dimension(50, 50));

            labelArray_2025[k].setHorizontalAlignment(SwingConst
ants.CENTER);

            panelArray_2025.add(labelArray_2025[k]);
        }

        mergeQueue_2025.clear();
        generateMergeSteps_2025(0, array_2025.length
- 1);
        stepButton_2025.setEnabled(true);
        stepArea_2025.setText("");
        stepCount_2025 = 1;
        isMerging_2025 = false;

        panelArray_2025.revalidate();
        panelArray_2025.repaint();
    }

```

Kode Program 2.67 *Method Set Array*

Method ini digunakan untuk mengambil input pengguna, mengubahnya menjadi *array* integer, menampilkan data pada panel visualisasi, serta menyiapkan proses *Merge Sort*.

9. Membuat *method generateMergeSteps_2025()*

```

private void generateMergeSteps_2025(int left, int
right) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    generateMergeSteps_2025(left, mid);
    generateMergeSteps_2025(mid + 1, right);
    mergeQueue_2025.add(new int[]{left, mid,
right});
}

```

Kode Program 2.68 *Generate Merge Steps*

Method ini digunakan untuk menentukan urutan proses penggabungan (*merge*) yang akan dilakukan selama algoritma *Merge Sort* berjalan.

10. Membuat *method performStep_2025()*

```
private void performStep_2025() {
    resetHighlights_2025();

    if (!isMerging_2025 &&
        !mergeQueue_2025.isEmpty()) {
        int[] range_2025 =
mergeQueue_2025.poll();
        left_2025 = range_2025[0];
        mid_2025 = range_2025[1];
        right_2025 = range_2025[2];
        temp_2025 = new int[right_2025 -
left_2025 + 1];
        i_2025 = left_2025;
        j_2025 = mid_2025 + 1;
        k_2025 = 0;
        copying_2025 = false;
        isMerging_2025 = true;
        stepArea_2025.append("Langkah " +
stepCount_2025++ +
                                ": Mulai merge dari " +
left_2025 + " ke " + right_2025 + "\n");
        return;
    }

    if (isMerging_2025 && !copying_2025) {
        if (i_2025 <= mid_2025 && j_2025 <=
right_2025) {

            labelArray_2025[i_2025].setBackground(Color.CYAN);
            labelArray_2025[j_2025].setBackground(Color.CYAN);
```

Kode Program 2.69 *Method Perform Step*

Method ini merupakan inti program yang menjalankan satu langkah *Merge Sort* setiap kali tombol “Langkah Selanjutnya” ditekan.

11. Membandingkan dan menggabungkan elemen

```

        if (array_2025[i_2025] <=
array_2025[j_2025]) {
            temp_2025[k_2025++] =
array_2025[i_2025++];
        } else {
            temp_2025[k_2025++] =
array_2025[j_2025++];
        }
        stepArea_2025.append("Langkah "
+ stepCount_2025++ + ": Bandingkan dan salin
elemen\n");
        return;
    } else if (i_2025 <= mid_2025) {
        temp_2025[k_2025++] =
array_2025[i_2025++];
        stepArea_2025.append("Langkah "
+ stepCount_2025++ + ": Salin sisa kiri\n");
        return;
    } else if (j_2025 <= right_2025) {
        temp_2025[k_2025++] =
array_2025[j_2025++];
        stepArea_2025.append("Langkah "
+ stepCount_2025++ + ": Salin sisa kanan\n");
        return;
    } else {
        copying_2025 = true;
        k_2025 = 0;
        return;
    }
}

```

Kode Program 2.70 Proses *Bubble Merge Sort*

Syntax ini digunakan untuk membandingkan elemen dari *sub-array* kiri dan kanan, kemudian menyalin elemen yang lebih kecil ke *array* sementara.

12. Menyalin hasil *merge* ke *array* utama

```

    if (copying_2025 && k_2025 <
temp_2025.length) {
        array_2025[left_2025 + k_2025] =
temp_2025[k_2025];
    }
}

```

```

        labelArray_2025[left_2025 +
k_2025].setText(String.valueOf(temp_2025[k_2025]));
        labelArray_2025[left_2025 +
k_2025].setBackground(Color.GREEN);
        k_2025++;
        stepArea_2025.append("Langkah " +
stepCount_2025++ + ": Tempelkan ke array utama\n");
        return;
    }

    if (copying_2025 && k_2025 ==
temp_2025.length) {
        isMerging_2025 = false;
        copying_2025 = false;
    }

    if (mergeQueue_2025.isEmpty() &&
!isMerging_2025) {
        stepArea_2025.append("Selesai.\n");
        stepButton_2025.setEnabled(false);
        JOptionPane.showMessageDialog(this,
"Merge Sort selesai!");
    }
}

```

Kode Program 2.71 *Update Tampilan*

Syntax ini digunakan untuk memindahkan hasil penggabungan dari *array* sementara ke *array* utama sehingga urutan data menjadi lebih teratur.

13. Membuat *method resetHighlights_2025()*

```

private void resetHighlights_2025() {
    if (labelArray_2025 == null) return;
    for (JLabel label : labelArray_2025) {
        label.setBackground(Color.WHITE);
    }
}

```

Kode Program 2.72 *Method Reset Highlight*

Method ini digunakan untuk mengembalikan warna seluruh label ke kondisi awal sebelum langkah berikutnya dijalankan.

14. Membuat *method reset_2025()*

```
private void reset_2025() {  
    inputField_2025.setText("");  
    panelArray_2025.removeAll();  
    panelArray_2025.revalidate();  
    panelArray_2025.repaint();  
    stepArea_2025.setText("");  
    stepButton_2025.setEnabled(false);  
    mergeQueue_2025.clear();  
    isMerging_2025 = false;  
    stepCount_2025 = 1;  
}
```

Kode Program 2.73 *Method Reset*

Method ini digunakan untuk menghapus data, membersihkan tampilan, dan mengembalikan program ke kondisi awal.

15. Membuat *method main()*

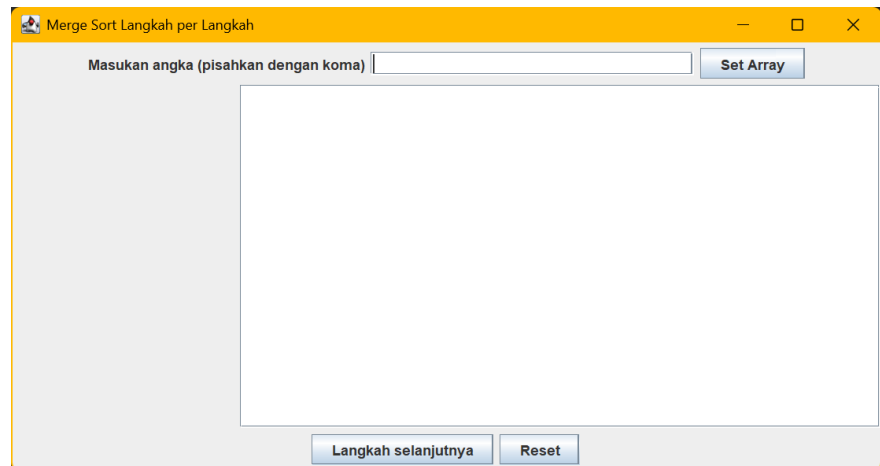
```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        MergeSortGUI_2511532025 gui = new  
MergeSortGUI_2511532025();  
        gui.setVisible(true);  
    });  
}
```

Kode Program 2.74 *Method Main*

Method utama digunakan untuk menjalankan aplikasi *GUI Merge Sort* dan menampilkan jendela program kepada pengguna.

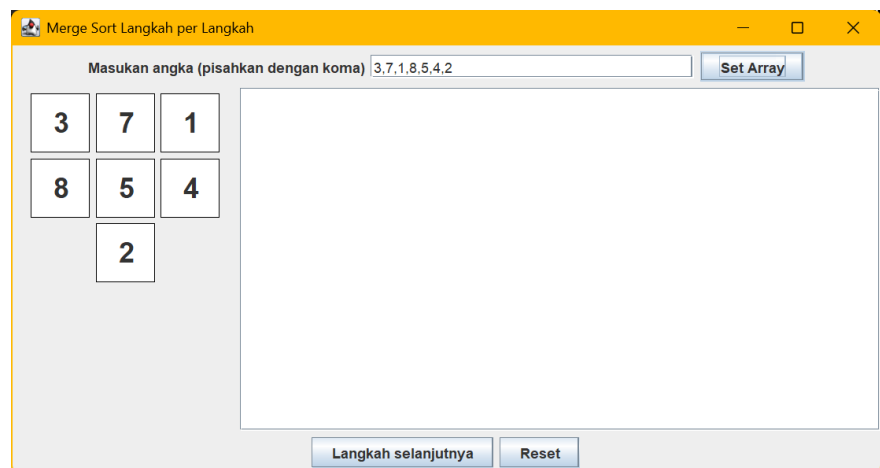
2.5.2 Hasil Program

Tampilan awal program



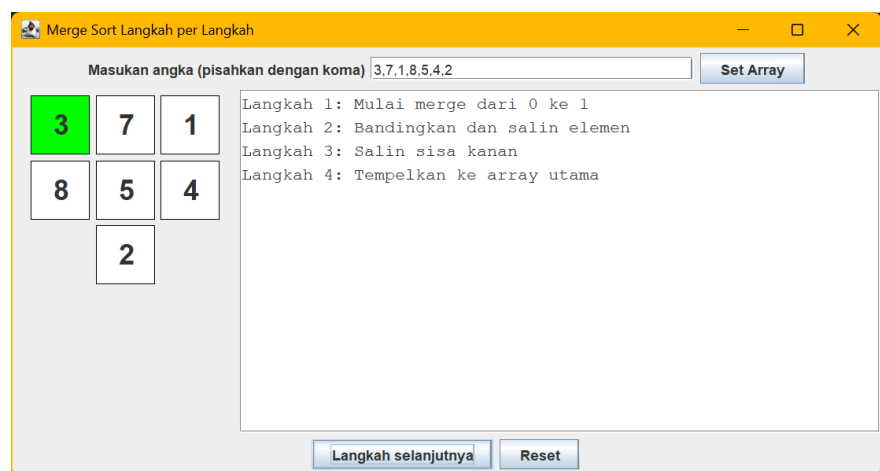
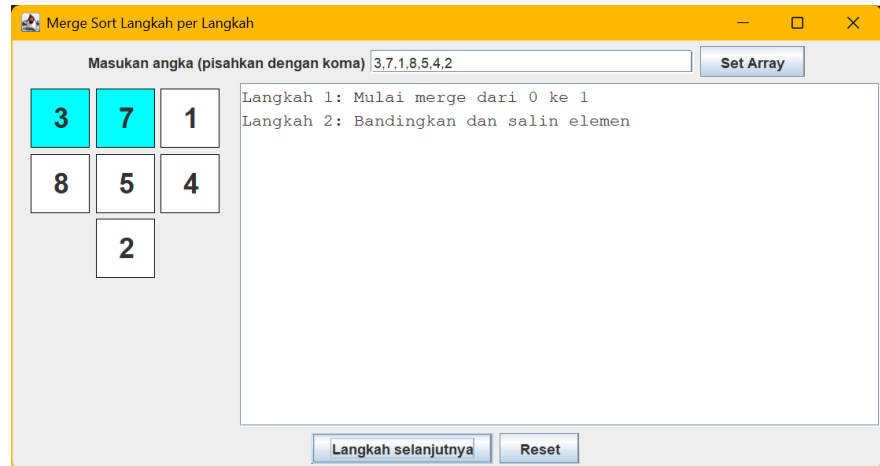
Gambar 2.8 Hasil Program Tampilan Awal Program
MergeSortGUI

Input data *array*



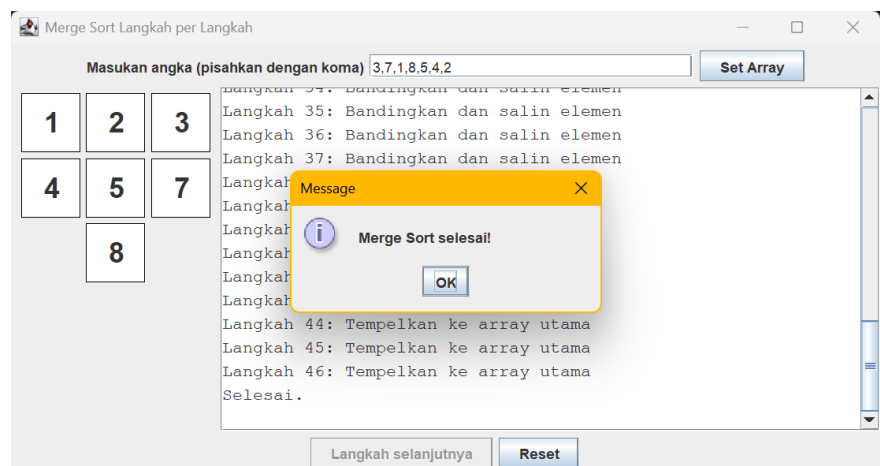
Gambar 2.9 Tampilan Data Sebelum Pengurutan

Proses *Bubble Sort* berlangsung



Gambar 2.10 Hasil Proses Perbandingan Data

Hasil akhir *sorting*



Gambar 2.11 Hasil Hasil Pengurutan Data Menggunakan *Merge Sort*

Berdasarkan hasil eksekusi program, pengguna dapat memasukkan data berupa angka yang dipisahkan dengan tanda koma. Setelah tombol “Set Array” ditekan, data ditampilkan dalam bentuk label pada panel visualisasi. Tombol “Langkah Selanjutnya” digunakan untuk menjalankan proses *Merge Sort* secara bertahap sehingga setiap proses penggabungan data dapat diamati dengan jelas.

Setelah seluruh langkah selesai dijalankan, *array* berhasil diurutkan dari nilai terkecil ke terbesar. Program juga menampilkan *log* setiap langkah penggabungan data sehingga pengguna dapat memahami cara kerja algoritma *Merge Sort* secara visual.

BAB III

KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma *sorting* merupakan metode yang digunakan untuk mengurutkan data agar tersusun secara teratur sehingga memudahkan proses pencarian, pengolahan, dan analisis data. Pada praktikum ini digunakan beberapa algoritma *sorting*, yaitu *Insertion Sort*, *Shell Sort*, *Quick Sort*, dan *Merge Sort* yang diimplementasikan menggunakan bahasa Java.

Melalui program *InsertionSort_2511532025*, dapat dipahami bahwa algoritma *Insertion Sort* bekerja dengan cara menyisipkan setiap elemen ke posisi yang sesuai pada bagian *array* yang telah terurut. Proses ini dilakukan dengan membandingkan dan menggeser elemen hingga diperoleh urutan yang benar.

Pada program *ShellSort_2511532025*, proses pengurutan dilakukan dengan membandingkan elemen-elemen yang memiliki jarak tertentu (*gap*). Nilai *gap* kemudian diperkecil secara bertahap hingga menjadi satu, sehingga proses pengurutan dapat dilakukan lebih efisien dibandingkan *Insertion Sort* pada jumlah data yang lebih besar.

Selanjutnya, program *QuickSort_2511532025* menunjukkan bahwa algoritma *Quick Sort* bekerja dengan memilih sebuah *pivot*, kemudian membagi data menjadi dua bagian berdasarkan nilai *pivot* tersebut. Proses pembagian dan pengurutan dilakukan secara rekursif hingga seluruh data berhasil diurutkan.

Pada program *MergeSort_2511532025*, algoritma *Merge Sort* bekerja dengan membagi array menjadi beberapa bagian yang lebih kecil, kemudian menggabungkannya kembali secara terurut melalui proses

merge. Metode ini memanfaatkan teknik *divide and conquer* sehingga mampu mengurutkan data secara efektif.

Selain program berbasis *console*, praktikum ini juga mengimplementasikan *Java GUI (Swing)* melalui program *BubbleSortGUI_2511532025* dan *MergeSortGUI_2511532025*. Kedua program tersebut mampu menampilkan proses *sorting* secara visual dan interaktif sehingga pengguna dapat mengamati setiap langkah pengurutan data secara bertahap. Visualisasi ini membantu dalam memahami cara kerja algoritma *Bubble Sort* dan *Merge Sort* dengan lebih jelas.

Dengan demikian, praktikum ini membantu memahami konsep dasar berbagai algoritma sorting, cara kerja masing-masing algoritma, serta implementasinya menggunakan Java berbasis *console* maupun *GUI*. Praktikum ini juga meningkatkan kemampuan dalam merancang program pengurutan data yang terstruktur, interaktif, dan mudah dipahami.

DAFTAR PUSTAKA

Wahyudi, S.T., M.T., *Struktur Data: Part 10 Sorting Algorithm*. Padang: Universitas Andalas, 2025.

Wahyudi, S.T., M.T., *Struktur Data: Part 11 Sorting Algorithm*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 8: Struktur Data

Topik: Algoritma Sorting (Shell, Quick, dan Merge)

Tema: Mengurutkan Playlist Lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma *sorting* untuk mengurutkan data lagu dalam playlist. Pilih salah satu: *Shell Sort*, *Quick Sort*, atau *Merge Sort*. Data lagu disimpan dalam *array* (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A - Z) atau Durasi (terpendek ke terpanjang).

2. Kode Program

```
package pekan8_2511532025;

class Lagu_2025 {
    String judul_2025;
    String penyanyi_2025;
    int durasi_2025;

    public Lagu_2025(String judul_2025, String penyanyi_2025, int
durasi_2025) {
        this.judul_2025 = judul_2025;
        this.penyanyi_2025 = penyanyi_2025;
        this.durasi_2025 = durasi_2025;
    }
}

public class Sorting_2511532025 {

    Lagu_2025[] dataLagu_2025 = new Lagu_2025[20];
    int jumlahData_2025 = 0;

    public void inputData_2025() {
        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Risk It All", "Bruno Mars",
233);

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Never Let Go", "Cortis", 215);

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Superpower", "Daniel Caesar",
248);

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Perfect", "Ed Sheeran", 263);
    }
}
```

```

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Photograph", "Ed Sheeran", 258);

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Yellow", "Coldplay", 269);

        dataLagu_2025[jumlahData_2025++] =
            new Lagu_2025("Let Me Go", "Daniel Caesar",
221);
    }

    public int partition_2025(int low_2025, int high_2025) {

        int pivot_2025 = dataLagu_2025[high_2025].durasi_2025;
        int i_2025 = low_2025 - 1;

        for (int j_2025 = low_2025; j_2025 < high_2025; j_2025++)
        {

            if (dataLagu_2025[j_2025].durasi_2025 < pivot_2025) {

                i_2025++;

                Lagu_2025 temp_2025 = dataLagu_2025[i_2025];
                dataLagu_2025[i_2025] = dataLagu_2025[j_2025];
                dataLagu_2025[j_2025] = temp_2025;

            }

        }

        Lagu_2025 temp_2025 = dataLagu_2025[i_2025 + 1];
        dataLagu_2025[i_2025 + 1] = dataLagu_2025[high_2025];
        dataLagu_2025[high_2025] = temp_2025;

        return i_2025 + 1;

    }

    public void quickSort_2025(int low_2025, int high_2025) {

        if (low_2025 < high_2025) {

            int pi_2025 = partition_2025(low_2025, high_2025);

            quickSort_2025(low_2025, pi_2025 - 1);
            quickSort_2025(pi_2025 + 1, high_2025);

        }

    }

    public void tampilData_2025() {

        for (int i_2025 = 0; i_2025 < jumlahData_2025; i_2025++)
        {

            System.out.println(
                (i_2025 + 1) + ". "
                + dataLagu_2025[i_2025].judul_2025
                + " - "

```

```

        + dataLagu_2025[i_2025].durasi_2025
        + " detik");
    }
}

public static void main(String[] args) {

    Sorting_2511532025 obj_2025 =
        new Sorting_2511532025();

    obj_2025.inputData_2025();

    System.out.println("=== Sorting Playlist NIM: 2511532025
===");

    System.out.println("\nData Sebelum Sorting:");
    obj_2025.tampilData_2025();

    obj_2025.quickSort_2025(
        0,
        obj_2025.jumlahData_2025 - 1);

    System.out.println(
        "\nData Setelah Quick Sort (Durasi Asc):");

    obj_2025.tampilData_2025();
}
}

```

Quick Sort bekerja dengan memilih satu data sebagai *pivot*, lalu membagi data menjadi dua bagian berdasarkan nilai *pivot* tersebut. Proses ini dilakukan secara berulang sampai semua data tersusun dengan urut. Pada program ini, *Quick Sort* digunakan untuk mengurutkan playlist lagu berdasarkan durasi dari yang paling pendek hingga paling panjang. Selain mudah dipahami, algoritma ini juga cukup efisien untuk mengurutkan data dalam jumlah yang banyak.

Kompleksitas Algoritma *Quick Sort*

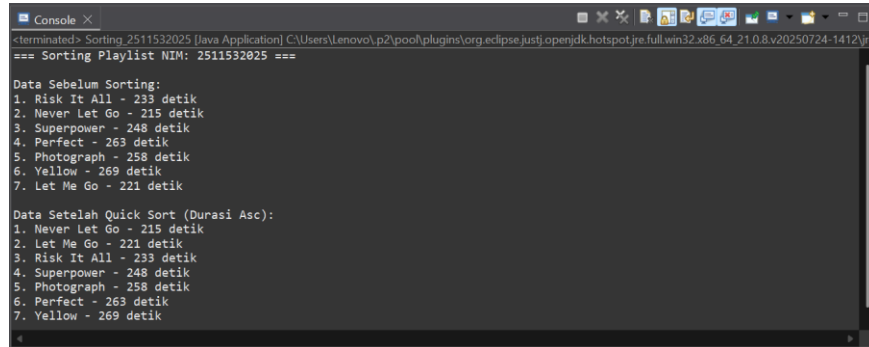
Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n^2)$

Pada kondisi normal *Quick Sort* dapat mengurutkan data dengan cukup cepat. Namun pada kondisi tertentu, seperti ketika pemilihan *pivot* kurang tepat, waktu prosesnya bisa menjadi lebih lama.

3. Tampilan Output Program



```
<terminated> Sorting_2511532025 [Java Application] C:\Users\Lenovo\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre...
=== Sorting Playlist NIM: 2511532025 ===

Data Sebelum Sorting:
1. Risk It All - 233 detik
2. Never Let Go - 215 detik
3. Superpower - 248 detik
4. Perfect - 263 detik
5. Photograph - 258 detik
6. Yellow - 269 detik
7. Let Me Go - 221 detik

Data Setelah Quick Sort (Durasi Asc):
1. Never Let Go - 215 detik
2. Let Me Go - 221 detik
3. Risk It All - 233 detik
4. Superpower - 248 detik
5. Photograph - 258 detik
6. Perfect - 263 detik
7. Yellow - 269 detik
```